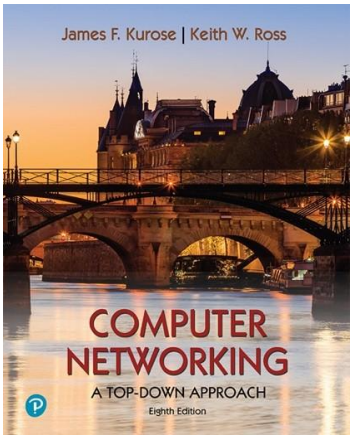
Wireshark Lab: HTTP v8.0

Supplement to *Computer Networking: A Top-Down Approach, 8th ed.*, J.F. Kurose and K.W. Ross

"Tell me and I forget. Show me and I remember. Involve me and I understand." Chinese proverb

© 2005-2020, J.F Kurose and K.W. Ross, All Rights Reserved



Integrity statement to be submitted with your assignment:
Sources of help that I used: (list all sources and/or names of people who helped, or write None.)
The answers and captures below were captured/written by me and not copied from the help sources.

Having gotten our feet wet with the Wireshark packet sniffer in the introductory lab, we're now ready to use Wireshark to investigate protocols in operation. In this lab, we'll explore several aspects of the HTTP protocol: the basic GET/response interaction, HTTP message formats, retrieving large HTML files, and retrieving HTML files with embedded objects. You are required to answer 14 questions from these four parts, and to attach captures or screenshots that support your answers.

1. The Basic HTTP GET/response interaction

Let's begin our exploration of HTTP by downloading a very simple HTML file - one that is very short, and contains no embedded objects. Do the following:

- Start up your web browser.
- Start up the Wireshark packet sniffer, as described in the Introductory lab (but don't yet begin packet capture). Enter "http" (just the letters, not the quotation marks) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window. (We're only interested in the HTTP protocol here, and don't want to see the clutter of all captured packets).
- Wait a bit more than one minute (we'll see why shortly), and then begin Wireshark packet capture.
- Enter the following to your browser
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html
  Your browser should display the very simple, one-line HTML file.
- Stop Wireshark packet capture.

---

[1] References to figures and sections are for the 8th edition of our text, *Computer Networks, A Top-down Approach, 8h ed., J.F. Kurose and K.W. Ross, Addison-Wesley/Pearson, 2020.*

Your Wireshark window should look similar to the window shown in Figure 1.

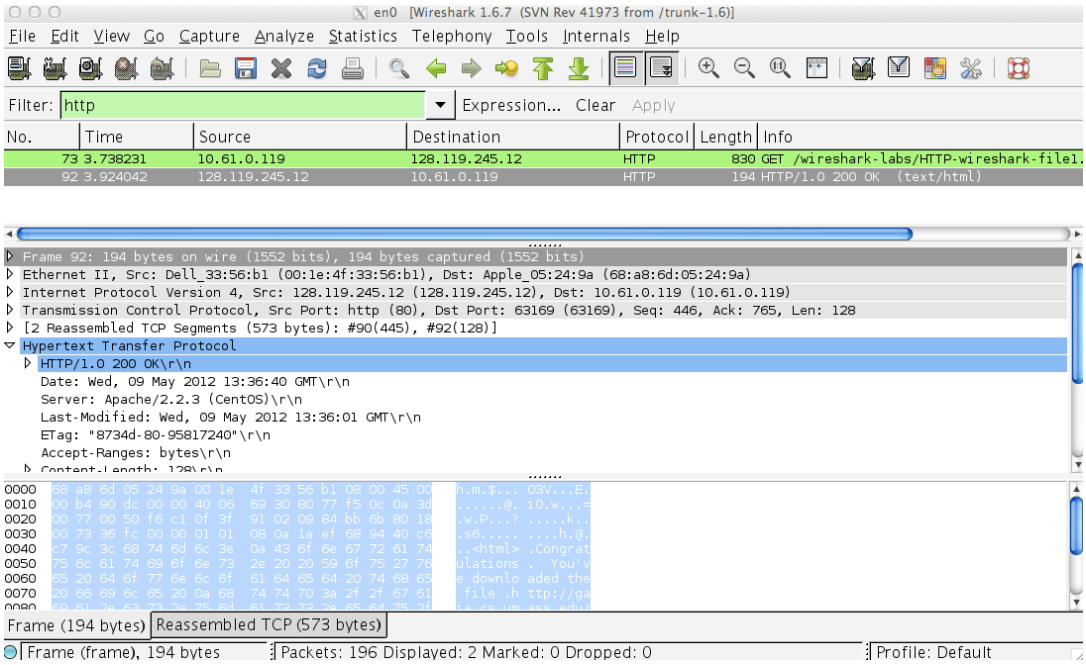


Figure 1: Wireshark Display after http://gaia.cs.umass.edu/wireshark-labs/ HTTP-wireshark-file1.html has been retrieved by your browser

The example in Figure 1 shows in the packet-listing window that two HTTP messages were captured: the GET message (from your browser to the gaia.cs.umass.edu web server) and the response message from the server to your browser. The packet-contents window shows details of the selected message (in this case the HTTP OK message, which is highlighted in the packet-listing window). Recall that since the HTTP message was carried inside a TCP segment, which was carried inside an IP datagram, which was carried within an Ethernet frame, Wireshark displays the Frame, Ethernet, IP, and TCP packet information as well. We want to minimize the amount of non-HTTP data displayed (we're interested in HTTP here, and will be investigating these other protocols is later labs), so make sure the boxes at the far left of the Frame, Ethernet, IP and TCP information have a plus sign or a right-pointing triangle (which means there is hidden, undisplayed information), and the HTTP line has a minus sign or a down-pointing triangle (which means that all information about the HTTP message is displayed).

By looking at the information in the HTTP GET and response messages, answer the following questions. When answering the following questions, you should print out the GET and response messages (select the packet to be printed, then go to file→print and select the options: "selected packet only" and "as displayed" after making sure that you expanded/collapsed the details in the contents window as mentioned earlier), and indicate where in the message you've found the information that answers the following questions.

1. **What version of HTTP is your browser running? What version of HTTP is theserver running?**
2. **What languages (if any) does your browser indicate that it can accept to the server?**
3. **What is the IP address of your computer? Of the gaia.cs.umass.edu server?**
4. **What is the status code returned from the server to your browser?**
5. **When was the HTML file that you are retrieving last modified at the server?**
6. **How many bytes of content are being returned to your browser?**
7. **How long did it take from when the HTTP GET message was sent until the HTTP response was received? (By default, the value of the Time column in the packet-listing window is the amount of time, in seconds, since Wireshark tracing began. To display the Time field in time-of-day format, select the Wireshark *View* pull down menu, then select Time *Display Format,* then select *Time-of-day*.)**

In your answer to question 5 above, you might have been surprised to find that the document you just retrieved was last modified within a minute before you downloaded the document. That's because (for this particular file), the gaia.cs.umass.edu server is setting the file's last-modified time to be the current time, and is doing so once per minute. Thus, if you wait a minute between accesses, the file will appear to have been recently modified, and hence your browser will download a "new" copy of the document.

2. The HTTP CONDITIONAL GET/response interaction

Recall from Section 2.2.5 of the text, that most web browsers perform object caching and thus perform a conditional GET when retrieving an HTTP object. Before performing the steps below, make sure your browser's cache is empty. (To do this under Firefox, select *Tools->Clear Recent History* and check the Cache box, or for Internet Explorer, select *Tools->Internet Options->Delete File;* similarly for Chrome, you can select *History* (Ctrl-H) and then *clear browsing data*. These actions will remove cached files from your browser's cache.) Now do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html
  Your browser should display a very simple five-line HTML file.

- Quickly enter the same URL into your browser again (or simply select the refresh button on your browser)
- Stop Wireshark packet capture, and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.

Answer the following questions (again you need to print the corresponding packets and show how you obtained your answers from the captures or packet details):

8. **Inspect the contents of the first HTTP GET request from your browser to theserver. Do you see an "IF-MODIFIED-SINCE" line in the HTTP GET?**
9. **Inspect the contents of the server response. Did the server explicitly return thecontents of the file? How can you tell?**
10. **Now inspect the contents of the second HTTP GET request from your browser tothe server. Do you see an "IF-MODIFIED-SINCE:" line in the HTTP GET? If so, what information follows the "IF-MODIFIED-SINCE:" header?**
11. **What is the HTTP status code and phrase returned from the server in response to this second HTTP GET? Did the server explicitly return the contents of the file?Explain.**

3. Retrieving Long Documents

In our examples thus far, the documents retrieved have been simple and short HTML files. Let's next see what happens when we download a long HTML file.

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html
  Your browser should display the rather lengthy US Bill of Rights.
- Stop Wireshark packet capture, and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed.

In the packet-listing window, you should see your HTTP GET message, followed by a multiple-packet TCP response to your HTTP GET request. This multiple-packet response deserves a bit of explanation. Recall from Section 2.2 (see Figure 2.9 in the text) that the HTTP response message consists of a status line, followed by header lines, followed by a blank line, followed by the entity body. In the case of our HTTP GET, the

entity body in the response is the *entire* requested HTML file. In our case here, the HTML file is rather long, and at 4500 bytes is too large to fit in one TCP packet. The single HTTP response message is thus broken into several pieces by TCP, with each piece being contained within a separate TCP segment (see Figure 1.24 in the text).

12. **How many HTTP GET request messages did your browser send?**

13. **Inspect the contents of the HTTP response message and obtain the number of data-containing TCP segments that were needed to carry the single HTTPresponse and the text of the Bill of Rights?**

4. HTML Documents with Embedded Objects

Now that we've seen how Wireshark displays the captured packet traffic for large HTML files, we can look at what happens when your browser downloads a file with embedded objects, i.e., a file that includes other objects (in the example below, image files) that are stored on another server(s).

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html
  Your browser should display a short HTML file with two images. These two images are referenced in the base HTML file. That is, the images themselves are not contained in the HTML; instead the URLs for the images are contained in the downloaded HTML file. As discussed in the textbook, your browser will have to retrieve these logos from the indicated web sites. Our publisher's logo is retrieved from the gaia.cs.umass.edu web site. The image of the cover for our 5th edition (one of our favorite covers) is stored at the caite.cs.umass.edu server. (These are two different web servers inside cs.umass.edu).
- Stop Wireshark packet capture, and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed.

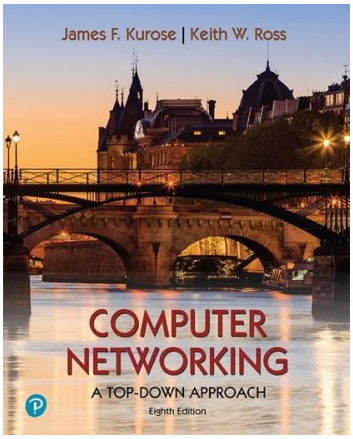
14. **How many HTTP GET request messages did your browser send? To which Internet addresses were these GET requests sent?**

Wireshark Lab: HTTP v8.0

Supplement to *Computer Networking: A Top-Down Approach, 8th ed.*, J.F. Kurose and K.W. Ross

"Tell me and I forget. Show me and I remember. Involve me and I understand." Chinese proverb

© 2005-2020, J.F Kurose and K.W. Ross, All Rights Reserved



Integrity statement to be submitted with your assignment:
Sources of help that I used: (list all sources and/or names of people who helped, or write None.)
The answers and captures below were captured/written by me and not copied from the help sources.

Having gotten our feet wet with the Wireshark packet sniffer in the introductory lab, we're now ready to use Wireshark to investigate protocols in operation. In this lab, we'll explore several aspects of the HTTP protocol: the basic GET/response interaction, HTTP message formats, retrieving large HTML files, and retrieving HTML files with embedded objects. You are required to answer 14 questions from these four parts, and to attach captures or screenshots that support your answers.

1. The Basic HTTP GET/response interaction

Let's begin our exploration of HTTP by downloading a very simple HTML file - one that is very short, and contains no embedded objects. Do the following:

- Start up your web browser.
- Start up the Wireshark packet sniffer, as described in the Introductory lab (but don't yet begin packet capture). Enter "http" (just the letters, not the quotation marks) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window. (We're only interested in the HTTP protocol here, and don't want to see the clutter of all captured packets).
- Wait a bit more than one minute (we'll see why shortly), and then begin Wireshark packet capture.
- Enter the following to your browser
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html
  Your browser should display the very simple, one-line HTML file.
- Stop Wireshark packet capture.

---

[1] References to figures and sections are for the 8th edition of our text, *Computer Networks, A Top-down Approach, 8h ed., J.F. Kurose and K.W. Ross, Addison-Wesley/Pearson, 2020.*

Your Wireshark window should look similar to the window shown in Figure 1.

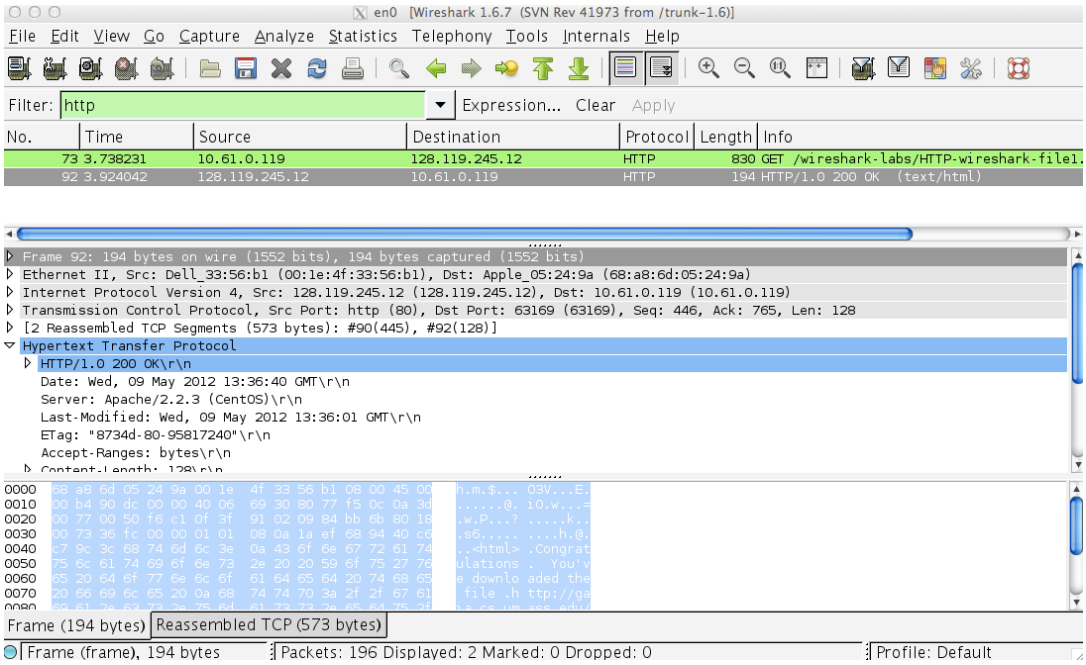


Figure 1: Wireshark Display after http://gaia.cs.umass.edu/wireshark-labs/ HTTP-wireshark-file1.html has been retrieved by your browser

The example in Figure 1 shows in the packet-listing window that two HTTP messages were captured: the GET message (from your browser to the gaia.cs.umass.edu web server) and the response message from the server to your browser. The packet-contents window shows details of the selected message (in this case the HTTP OK message, which is highlighted in the packet-listing window). Recall that since the HTTP message was carried inside a TCP segment, which was carried inside an IP datagram, which was carried within an Ethernet frame, Wireshark displays the Frame, Ethernet, IP, and TCP packet information as well. We want to minimize the amount of non-HTTP data displayed (we're interested in HTTP here, and will be investigating these other protocols is later labs), so make sure the boxes at the far left of the Frame, Ethernet, IP and TCP information have a plus sign or a right-pointing triangle (which means there is hidden, undisplayed information), and the HTTP line has a minus sign or a down-pointing triangle (which means that all information about the HTTP message is displayed).

By looking at the information in the HTTP GET and response messages, answer the following questions. When answering the following questions, you should print out the GET and response messages (select the packet to be printed, then go to file→print and select the options: "selected packet only" and "as displayed" after making sure that you expanded/collapsed the details in the contents window as mentioned earlier), and indicate where in the message you've found the information that answers the following questions.

Sample HTTP request and response messages are printed below, and the parts needed for the respective questions are highlighted in colors that match the answers.

HTTP GET:

No. Time Source Destination Protocol Length Info
1296 09:20:51.284629 10.21.5.185 128.119.245.12 HTTP 638 GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1
Frame 1296: 638 bytes on wire (5104 bits), 638 bytes captured (5104 bits) on interface \Device\NPF_{0ECC3CF0-5572-48B1-A98A-FB26CDCC32F4}, id 0
Ethernet II, Src: IntelCor_65:01:b2 (18:26:49:65:01:b2), Dst: Cisco_f6:b7:00 (08:96:ad:f6:b7:00)
Internet Protocol Version 4, Src: 10.21.5.185, Dst: 128.119.245.12
Transmission Control Protocol, Src Port: 57277, Dst Port: 80, Seq: 1, Ack: 1, Len: 584
Hypertext Transfer Protocol
GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1\r\n
[Expert Info (Chat/Sequence): GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1\r\n]
[GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1\r\n]
[Severity level: Chat]
[Group: Sequence]
Request Method: GET
Request URI: /wireshark-labs/INTRO-wireshark-file1.html
Request Version: HTTP/1.1
Host: gaia.cs.umass.edu\r\n
Connection: keep-alive\r\n
Cache-Control: max-age=0\r\n
Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0.0.0 Safari/537.36\r\n
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signedexchange;v=b3;q=0.7\r\n
Accept-Encoding: gzip, deflate\r\n
Accept-Language: en-US,en;q=0.9\r\n
If-None-Match: "51-606c9645f08ee"\r\n
If-Modified-Since: Tue, 03 Oct 2023 05:45:02 GMT\r\n
\r\n
[Full request URI: http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html]
[HTTP request 1/1]
[Response in frame: 1300]

HTTP OK:

No. Time Source Destination Protocol Length Info
1300 09:20:51.430835 128.119.245.12 10.21.5.185 HTTP 492 HTTP/1.1 200 OK (text/html)
Frame 1300: 492 bytes on wire (3936 bits), 492 bytes captured (3936 bits) on interface \Device\NPF_{0ECC3CF0-5572-48B1-A98A-FB26CDCC32F4}, id 0
Ethernet II, Src: Cisco_f6:b7:00 (08:96:ad:f6:b7:00), Dst: IntelCor_65:01:b2 (18:26:49:65:01:b2)
Internet Protocol Version 4, Src: 128.119.245.12, Dst: 10.21.5.185
Transmission Control Protocol, Src Port: 80, Dst Port: 57277, Seq: 1, Ack: 585, Len: 438
Hypertext Transfer Protocol
HTTP/1.1 200 OK\r\n
[Expert Info (Chat/Sequence): HTTP/1.1 200 OK\r\n]
[HTTP/1.1 200 OK\r\n]
[Severity level: Chat]
[Group: Sequence]
Response Version: HTTP/1.1
Status Code: 200
[Status Code Description: OK]
Response Phrase: OK
Date: Tue, 03 Oct 2023 06:20:51 GMT\r\n
Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.33 mod_perl/2.0.11 Perl/v5.16.3\r\n

```
Last-Modified: Tue, 03 Oct 2023 05:59:02 GMT\r\n
ETag: "51-606c996708358"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 81\r\n
Keep-Alive: timeout=5, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=UTF-8\r\n
\r\n
[HTTP response 1/1]
[Time since request: 0.146206000 seconds]
[Request in frame: 1296]
[Request URI: http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html]
File Data: 81 bytes
Line-based text data: text/html (3 lines)
```

1. **What version of HTTP is your browser running? What version of HTTP is theserver running?**
   Both browser and server are running HTTP/1.1

2. **What languages (if any) does your browser indicate that it can accept to the server?**
   From the GET header field: *Accept-Language: en-US,en;q=0.9\r\n*, we can see that there are two accepted languages: **en-US** which is English as spoken in the United states and **en** which is English in general (q=0.9 attribute also assigns a preference factor of 0.9 to en-US as opposed to a preference of 0.1 for en)

3. **What is the IP address of your computer? Of the gaia.cs.umass.edu server?**
   IP of my computer: 10.21.5.185
   IP of gaia.cs.umass.edu: 128.119.245.12

4. **What is the status code returned from the server to your browser?**
   The status code is 200 as shown from the OK message capture:

5. **When was the HTML file that you are retrieving last modified at the server?**
   From the header in the OK message: *Last-Modified: Tue, 03 Oct 2023 05:59:02 GMT\r\n*, We see that it was last modified on the server side on Oct 03, 2023 at 05:59:02 GMT which is 08:59:02 AM Beirut time

6. **How many bytes of content are being returned to your browser?**
   81 bytes. This can be seen from the response header: *Content-Length: 81\r\n*

7. **How long did it take from when the HTTP GET message was sent until the HTTP response was received? (By default, the value of the Time column in the packet-listing window is the amount of time, in seconds, since Wireshark tracing began. To display the Time field in time-of-day format, select the Wireshark *View* pull down menu, then select Time *Display Format*, then select *Time-of-day*.)**
   As seen below, the time interval between the GET and the reply is
   2.883338 – 2.736579 = 0.146759 seconds

```
1495 08:45:02.736579   10.21.5.185      128.119.245.12   HTTP   612 GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1
1501 08:45:02.883338   128.119.245.12   10.21.5.185      HTTP   492 HTTP/1.1 200 OK  (text/html)
```

In your answer to question 5 above, you might have been surprised to find that the document you just retrieved was last modified within a minute before you downloaded the document. That's because (for this particular file), the gaia.cs.umass.edu server is setting the file's last-modified time to be the current time, and is doing so once per minute. Thus, if you wait a minute between accesses, the file will appear to have been recently modified, and hence your browser will download a "new" copy of the document.

2. The HTTP CONDITIONAL GET/response interaction

Recall from Section 2.2.5 of the text, that most web browsers perform object caching and thus perform a conditional GET when retrieving an HTTP object. Before performing the steps below, make sure your browser's cache is empty. (To do this under Firefox, select *Tools->Clear Recent History* and check the Cache box, or for Internet Explorer, select *Tools->Internet Options->Delete File;* similarly for Chrome, you can select *History* (Ctrl-H) and then *clear browsing data*. These actions will remove cached files from your browser's cache.) Now do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html
  Your browser should display a very simple five-line HTML file.

- Quickly enter the same URL into your browser again (or simply select the refresh button on your browser)
- Stop Wireshark packet capture, and enter “http” in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.

Answer the following questions (again you need to print the corresponding packets and show how you obtained your answers from the captures or packet details):

Printing the four messages for this part:

HTTP GET:

No. Time Source Destination Protocol Length Info
286 10:17:15.643667 10.21.5.185 128.119.245.12 HTTP 526 GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1
Frame 286: 526 bytes on wire (4208 bits), 526 bytes captured (4208 bits) on interface \Device\NPF_{0ECC3CF0-5572-48B1-A98A-FB26CDCC32F4}, id 0
Ethernet II, Src: IntelCor_65:01:b2 (18:26:49:65:01:b2), Dst: Cisco_f6:b7:00 (08:96:ad:f6:b7:00)
Internet Protocol Version 4, Src: 10.21.5.185, Dst: 128.119.245.12
Transmission Control Protocol, Src Port: 57980, Dst Port: 80, Seq: 1, Ack: 1, Len: 472
Hypertext Transfer Protocol
GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n
[Expert Info (Chat/Sequence): GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n]
[GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n]
[Severity level: Chat]
[Group: Sequence]
Request Method: GET
Request URI: /wireshark-labs/HTTP-wireshark-file2.html
Request Version: HTTP/1.1
Host: gaia.cs.umass.edu\r\n
Connection: keep-alive\r\n
Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0.0.0 Safari/537.36\r\n
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signedexchange;v=b3;q=0.7\r\n
Accept-Encoding: gzip, deflate\r\n
Accept-Language: en-US,en;q=0.9\r\n
\r\n
[Full request URI: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html]
[HTTP request 1/1]
[Response in frame: 290]

HTTP OK:

No. Time Source Destination Protocol Length Info
290 10:17:15.795135 128.119.245.12 10.21.5.185 HTTP 784 HTTP/1.1 200 OK (text/html)
Frame 290: 784 bytes on wire (6272 bits), 784 bytes captured (6272 bits) on interface \Device\NPF_{0ECC3CF0-5572-48B1-A98A-FB26CDCC32F4}, id 0
Ethernet II, Src: Cisco_f6:b7:00 (08:96:ad:f6:b7:00), Dst: IntelCor_65:01:b2 (18:26:49:65:01:b2)
Internet Protocol Version 4, Src: 128.119.245.12, Dst: 10.21.5.185
Transmission Control Protocol, Src Port: 80, Dst Port: 57980, Seq: 1, Ack: 473, Len: 730
Hypertext Transfer Protocol
HTTP/1.1 200 OK\r\n
[Expert Info (Chat/Sequence): HTTP/1.1 200 OK\r\n]
[HTTP/1.1 200 OK\r\n]
[Severity level: Chat]
[Group: Sequence]
Response Version: HTTP/1.1
Status Code: 200
[Status Code Description: OK]
Response Phrase: OK
Date: Tue, 03 Oct 2023 07:17:16 GMT\r\n
Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.33 mod_perl/2.0.11 Perl/v5.16.3\r\n
Last-Modified: Tue, 03 Oct 2023 05:59:02 GMT\r\n
ETag: "173-606c99670a680"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 371\r\n
Keep-Alive: timeout=5, max=100\r\n
Connection: Keep-Alive\r\n

Content-Type: text/html; charset=UTF-8\r\n
\r\n
[HTTP response 1/1]
[Time since request: 0.151468000 seconds]
[Request in frame: 286]
[Request URI: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html]
File Data: 371 bytes

HTTP Conditional GET

No. Time Source Destination Protocol Length Info
387 10:17:39.406087 10.21.5.185 128.119.245.12 HTTP 638 GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1
Frame 387: 638 bytes on wire (5104 bits), 638 bytes captured (5104 bits) on interface \Device\NPF_{0ECC3CF0-5572-48B1-A98A-FB26CDCC32F4}, id 0
Ethernet II, Src: IntelCor_65:01:b2 (18:26:49:65:01:b2), Dst: Cisco_f6:b7:00 (08:96:ad:f6:b7:00)
Internet Protocol Version 4, Src: 10.21.5.185, Dst: 128.119.245.12
Transmission Control Protocol, Src Port: 57981, Dst Port: 80, Seq: 1, Ack: 1, Len: 584
Hypertext Transfer Protocol
GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n
[Expert Info (Chat/Sequence): GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n]
[GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n]
[Severity level: Chat]
[Group: Sequence]
Request Method: GET
Request URI: /wireshark-labs/HTTP-wireshark-file2.html
Request Version: HTTP/1.1
Host: gaia.cs.umass.edu\r\n
Connection: keep-alive\r\n
Cache-Control: max-age=0\r\n
Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0.0.0 Safari/537.36\r\n
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signedexchange;v=b3;q=0.7\r\n
Accept-Encoding: gzip, deflate\r\n
Accept-Language: en-US,en;q=0.9\r\n
If-None-Match: "173-606c99670a680"\r\n
If-Modified-Since: Tue, 03 Oct 2023 05:59:02 GMT\r\n
\r\n
[Full request URI: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html]
[HTTP request 1/1]
[Response in frame: 391]

HTTP NOT MODIFIED:

No. Time Source Destination Protocol Length Info
391 10:17:39.552897 128.119.245.12 10.21.5.185 HTTP 294 HTTP/1.1 304 Not Modified
Frame 391: 294 bytes on wire (2352 bits), 294 bytes captured (2352 bits) on interface \Device\NPF_{0ECC3CF0-5572-48B1-A98A-FB26CDCC32F4}, id 0
Ethernet II, Src: Cisco_f6:b7:00 (08:96:ad:f6:b7:00), Dst: IntelCor_65:01:b2 (18:26:49:65:01:b2)
Internet Protocol Version 4, Src: 128.119.245.12, Dst: 10.21.5.185
Transmission Control Protocol, Src Port: 80, Dst Port: 57981, Seq: 1, Ack: 585, Len: 240
Hypertext Transfer Protocol
HTTP/1.1 304 Not Modified\r\n
[Expert Info (Chat/Sequence): HTTP/1.1 304 Not Modified\r\n]
[HTTP/1.1 304 Not Modified\r\n]
[Severity level: Chat]
[Group: Sequence]
Response Version: HTTP/1.1
Status Code: 304
[Status Code Description: Not Modified]
Response Phrase: Not Modified
Date: Tue, 03 Oct 2023 07:17:40 GMT\r\n
Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.33 mod_perl/2.0.11 Perl/v5.16.3\r\n
Connection: Keep-Alive\r\n
Keep-Alive: timeout=5, max=100\r\n
ETag: "173-606c99670a680"\r\n
\r\n
[HTTP response 1/1]
[Time since request: 0.146810000 seconds]
[Request in frame: 387]
[Request URI: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html

8. **Inspect the contents of the first HTTP GET request from your browser to the server. Do you see an "IF-MODIFIED-SINCE" line in the HTTP GET?**
   No there is no such field in the first GET message because the cache was empty at this point. So this GET message is not conditional on a last modified date.

9. **Inspect the contents of the server response. Did the server explicitly return thecontents of the file? How can you tell?**
   Yes, the server explicitly returned the content; we can see that from the OK status code, the content-length field, and the File data .

10. **Now inspect the contents of the second HTTP GET request from your browser tothe server. Do you see an "IF-MODIFIED-SINCE:" line in the HTTP GET? If so, what information follows the "IF-MODIFIED-SINCE:" header?**
    Yes, the second HTTP GET message includes this header:
    If-Modified-Since: Tue, 03 Oct 2023 05:59:02 GMT\r\n
    The value Tue, 03 Oct 2023 05:59:02 GMT matched the last modified field in the previous OK message, and it denotes the date of the last modification of the file from the previous get request.

11. **What is the HTTP status code and phrase returned from the server in response to this second HTTP GET? Did the server explicitly return the contents of the file?Explain.**
    Status code: 304 (Not modified)
    The server did not explicitly return the contents in response to the second HTTP GET because the content had not been modified and the client already had an up-to-date chached copy.

3. Retrieving Long Documents

In our examples thus far, the documents retrieved have been simple and short HTML files. Let's next see what happens when we download a long HTML file.

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html
  Your browser should display the rather lengthy US Bill of Rights.
- Stop Wireshark packet capture, and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed.

In the packet-listing window, you should see your HTTP GET message, followed by a multiple-packet TCP response to your HTTP GET request. This multiple-packet response deserves a bit of explanation. Recall from Section 2.2 (see Figure 2.9 in the text) that the HTTP response message consists of a status line, followed by header lines, followed by a blank line, followed by the entity body. In the case of our HTTP GET, the

entity body in the response is the *entire* requested HTML file. In our case here, the HTML file is rather long, and at 4500 bytes is too large to fit in one TCP packet. The single HTTP response message is thus broken into several pieces by TCP, with each piece being contained within a separate TCP segment (see Figure 1.24 in the text).

12. How many HTTP GET request messages did your browser send?

1 HTTP GET message was sent. This is packet number 467 in the trace as shown below.

467	09:25:04.586063	10.169.31.247	128.119.245.12	HTTP	552 GET /wireshark-labs/HTTP-wireshark-file3.html HTTP/1.1
478	09:25:04.982553	128.119.245.12	10.169.31.247	HTTP	685 HTTP/1.1 200 OK (text/html)

13. Inspect the contents of the HTTP response message and obtain the number of data-containing TCP segments that were needed to carry the single HTTPresponse and the text of the Bill of Rights?

Selecting the HTTP response and viewing its details shows that it needed 6 TCP segments to carry the payload as shown below.

```
Internet Protocol Version 4, Src: 128.119.245.12, Dst: 10.169.31.247
Transmission Control Protocol, Src Port: 80, Dst Port: 50565, Seq: 4207, Ack: 499, Len: 631
[6 Reassembled TCP Segments (4837 bytes): #471(1386), #472(24), #474(1386), #475(24), #477(1386), #478(631)]
   [Frame: 471, payload: 0-1385 (1386 bytes)]
   [Frame: 472, payload: 1386-1409 (24 bytes)]
   [Frame: 474, payload: 1410-2795 (1386 bytes)]
   [Frame: 475, payload: 2796-2819 (24 bytes)]
   [Frame: 477, payload: 2820-4205 (1386 bytes)]
   [Frame: 478, payload: 4206-4836 (631 bytes)]
   [Segment count: 6]
   [Reassembled TCP length: 4837]
```

4. HTML Documents with Embedded Objects

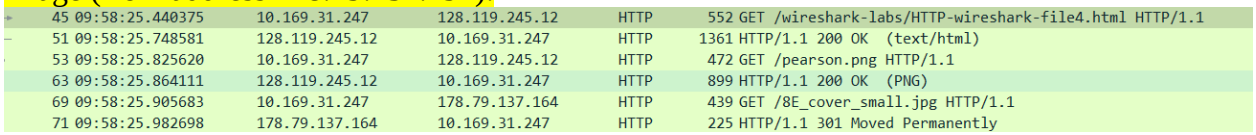
Now that we've seen how Wireshark displays the captured packet traffic for large HTML files, we can look at what happens when your browser downloads a file with embedded objects, i.e., a file that includes other objects (in the example below, image files) that are stored on another server(s).

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html
  Your browser should display a short HTML file with two images. These two images are referenced in the base HTML file. That is, the images themselves are not contained in the HTML; instead the URLs for the images are contained in the downloaded HTML file. As discussed in the textbook, your browser will have to retrieve these logos from the indicated web sites. Our publisher's logo is retrieved from the gaia.cs.umass.edu web site. The image of the cover for our 5th edition (one of our favorite covers) is stored at the caite.cs.umass.edu server. (These are two different web servers inside cs.umass.edu).
- Stop Wireshark packet capture, and enter "http" in the display-filter-specification

window, so that only captured HTTP messages will be displayed.

14. How many HTTP GET request messages did your browser send? To which Internet addresses were these GET requests sent?

As shown from the captures below, three HTTP GET messages were sent: one for the webpage HTML file (destination address is 128.119.245.12), one to retrieve the logo (pearson.png from the same address above), and a third one to retrieve the cover page image (from address 178.79.137.194).



45	09:58:25.440375	10.169.31.247	128.119.245.12	HTTP	552	GET /wireshark-labs/HTTP-wireshark-file4.html HTTP/1.1
51	09:58:25.748581	128.119.245.12	10.169.31.247	HTTP	1361	HTTP/1.1 200 OK (text/html)
53	09:58:25.825620	10.169.31.247	128.119.245.12	HTTP	472	GET /pearson.png HTTP/1.1
63	09:58:25.864111	128.119.245.12	10.169.31.247	HTTP	899	HTTP/1.1 200 OK (PNG)
69	09:58:25.905683	10.169.31.247	178.79.137.164	HTTP	439	GET /8E_cover_small.jpg HTTP/1.1
71	09:58:25.982698	178.79.137.164	10.169.31.247	HTTP	225	HTTP/1.1 301 Moved Permanently